

GitHub setup and course workflow

14.33 Research and Communication in Economics · Fall 2026

Git tracks changes to files on your computer. GitHub stores a remote copy of the repository and supports collaboration, issues, and links to evidence. These instructions use Git in a terminal; GitHub Desktop is not required.

1. Create a repository from the course template

1. Create a free account at github.com, or sign in to your existing account.
2. Open the [14.33 project template](#).
3. Select **Use this template** → **Create a new repository**.
4. Select your personal GitHub account as the owner.
5. Name the repository `fa26-1433-YOUR-KERBEROS`. Replace `YOUR-KERBEROS` with your MIT Kerberos username.
6. Use the repository visibility specified in class, then select **Create repository**.
7. In your new repository, select **Settings** → **Collaborators**. Add the instructor and TA using the GitHub usernames provided in class.

This access is required by the [course syllabus](#).

Your new repository contains the course directory structure and README files. It is separate from the original template. Do not commit confidential data, passwords, API keys, or other secrets.

2. Install Git and authentication support

Windows

Install [Git for Windows](#). Keep Git Credential Manager selected during installation.

macOS

If you use [Homebrew](#), run:

```
brew install git
brew install --cask git-credential-manager
```

Git Credential Manager opens a browser when GitHub authentication is required. Follow the browser prompts. See [GitHub's authentication instructions](#) if authentication fails.

Identify your commits

Run these commands once. Replace the example values with your name and the email address associated with GitHub.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

3. Clone the repository

On the GitHub repository page, select **Code** → **HTTPS**, then copy the URL. In a terminal, run:

```
git clone REPOSITORY-HTTPS-URL
cd fa26-1433-YOUR-KERBEROS
```

Cloning creates a local project directory and connects it to the GitHub repository.

4. Save work with Git

Use this cycle whenever you reach a meaningful checkpoint:

```
git pull --ff-only
git status
git add path/to/file
git diff --staged
git commit -m "Describe the completed change"
git push
```

- `git pull --ff-only` downloads new remote commits without creating an unintended merge commit.
- `git status` shows changed, staged, and untracked files.
- `git add path/to/file` stages a specific file for the next commit.
- `git diff --staged` shows exactly what the next commit will contain.
- `git commit` records the staged changes locally.
- `git push` sends local commits to GitHub.

Use short commit messages that describe completed work, such as `Add baseline summary statistics`. Commit source code, documentation, and small derived outputs needed to understand the project. Do not commit large raw datasets unless the course staff approves them.

5. Use GitHub issues for the course

Use two types of issues:

- A **meeting agenda issue** summarizes the project before one scheduled meeting. Create a new agenda for each meeting.
- A **work issue** tracks one consequential decision, question, blocker, or incomplete task. Keep the issue open across meetings until you resolve it.

Create a meeting agenda issue

At least 24 hours before each one-on-one meeting:

1. Select **Issues** → **New issue** in your project repository.
2. Title the issue `Meeting preparation – YYYY-MM-DD`.
3. Copy the template from the syllabus's **Individual meeting preparation** section into the issue.
4. Complete each section. Aim for approximately 250 words, excluding links and tables.
5. Link each finding to supporting code, output, or documentation in the repository.
6. In **Decisions, questions, or blockers**, link each relevant work issue by typing its number, such as `#23`.

Report findings and decisions rather than time spent. Use the agenda as a concise guide for the meeting, not as the complete history of the project.

Create and link a work issue

Create a separate work issue when an item needs evidence, discussion, or follow-up beyond one meeting:

1. Give the issue a specific title, such as `Choose the appropriate sample definition`.
2. State the decision, question, blocker, or task.
3. Add the relevant evidence, alternatives, and your current recommendation.
4. State what must be true before you can close the issue.
5. Link the work issue from the next meeting agenda by typing its issue number.

You may test important alternatives on short-lived branches. Link each branch from the applicable work issue. After the meeting, update the agenda with agreed next steps. Record each decision and its rationale in the applicable work issue, then close that issue when the work is complete.

6. Correct mistakes without deleting history

If a change has not been committed, edit the file again. To remove a file from the staging area without discarding its changes, run:

```
git restore --staged path/to/file
```

If an incorrect commit is already on GitHub, preserve the shared history. Either make a correcting commit or create a reversal commit:

```
git revert COMMIT-ID
git push
```

Ask the instructor or TA before rewriting published history with commands such as `git reset --hard` or a force push.